

□ PNG LSB-Стеганография — Постановка задачи и описание алгоритма

Постановщик задачи / Разработчик алгоритма: Гаджигасанов Тимур (soulmier)

Проект: [steganography](#)

Версия документа: 1.0

Статус проекта: 1.0

Язык реализации: Python 3.13, PyQt5

Содержание

1. [Введение и общая концепция](#)
 2. [Что такое стеганография](#)
 3. [Постановка задачи](#)
 4. [Технические ограничения и требования к входным данным](#)
 5. [Детальное описание алгоритма LSB в палитре PNG](#)
 6. [Формат встраиваемых данных](#)
 7. [Алгоритм встраивания](#)
 8. [Алгоритм извлечения](#)
 9. [Расчёт ёмкости канала](#)
 10. [Архитектура приложения](#)
 11. [Описание тестового покрытия](#)
 12. [Глоссарий](#)
-

1. Введение и общая концепция

Данный документ описывает концептуальную и алгоритмическую основу учебного проекта по **стеганографии** — науке о скрытой передаче информации. Документ охватывает:

- обоснование выбора предметной области и метода,
- формальное описание алгоритма встраивания и извлечения данных,
- технические ограничения и расчёты ёмкости,
- архитектурные решения, принятые в ходе проектирования,
- полное описание тестового покрытия.

Общая идея приложения

Приложение представляет собой **кроссплатформенное десктопное GUI-приложение** на

Python + PyQt5, которое позволяет пользователю:

1. Загрузить PNG-изображение с диска.
2. Ввести текстовое сообщение.
3. Незаметно спрятать сообщение внутри изображения.
4. Сохранить «стего-изображение» — файл, внешне неотличимый от оригинала.
5. В любой момент извлечь спрятанное сообщение из файла.

Соккрытие происходит **исключительно в палитре PNG** (чанк PLTE), без изменения пиксельных данных изображения.

2. Что такое стеганография

Стеганография (от греч. *στεγανός* — скрытый, *γράφω* — пишу) — это искусство и наука о скрытой передаче информации путём сокрытия самого факта передачи. В отличие от **криптографии**, которая скрывает *содержимое* сообщения (шифрование), стеганография скрывает *существование* сообщения.

Сравнение подходов

Свойство	Криптография	Стеганография
Что скрывается	Содержание сообщения	Сам факт существования сообщения
Обнаружимость	Факт передачи очевиден	Факт передачи скрыт
Защита при обнаружении	Высокая (ключ неизвестен)	Низкая (после обнаружения данные читаемы)
Типичный носитель	Любой канал	Мультимедиа (изображения, аудио, видео)
Сочетаемость	Можно комбинировать	Можно комбинировать с шифрованием

Область применения стеганографии

- **Цифровые водяные знаки** — защита авторских прав (DRM).
- **Скрытые каналы связи** — передача данных в условиях цензуры.
- **Идентификация источника утечки** — отпечатки (fingerprinting).
- **Образовательные цели** — изучение методов обработки изображений.

В данном учебном проекте стеганография реализована **без шифрования**: сообщение встраивается в открытом виде.

3. Постановка задачи

3.1 Формулировка задачи

Разработать программный инструмент, реализующий **метод LSB (Least Significant Bit)** для скрытия текстовых сообщений в **палитре PNG-изображений** (чанк PLTE, тип цвета ColorType = 3).

3.2 Функциональные требования

№	Требование
01	Загрузка PNG-изображения из файловой системы
02	Автоматическое приведение любого PNG к формату Indexed8 (256-цветная палитра)
03	Встраивание текстового сообщения (UTF-8) в LSB-биты RGB-компонентов палитры
04	Извлечение скрытого сообщения из стего-изображения
05	Сохранение стего-изображения в PNG-файл с сохранением палитры
06	Визуальное сравнение исходного и стего-изображений
07	Отображение извлечённого сообщения в интерфейсе
08	Проверка ёмкости палитры до встраивания
09	Поддержка Unicode (кириллица, эмодзи и др.) через UTF-8
10	Информирование пользователя через статусную строку

3.3 Ограничения

- Метод работает **только с PNG-изображениями** (формат с поддержкой палитры PLTE).
 - Для встраивания обязательно наличие **цветовой палитры** (Format_Indexed8).
 - Максимальная длина сообщения ограничена **размером палитры**: 96 байт для стандартной 256-цветной палитры.
 - Формат сохранения — исключительно **PNG**.
-

4. Технические ограничения и требования к входным данным

4.1 Требования к изображению-носителю

Изображение должно быть в формате PNG. При загрузке нецветовой PNG (например, RGB24, RGBA32) приложение автоматически конвертирует его в `Format_Indexed8` через алгоритм квантования цветов Qt (**медианное разрезание**, до 256 цветов).

Рекомендуемые параметры изображения:

- Формат файла: `.png`
- Тип цвета: `ColorType = 3` (Indexed color, 256-цветная палитра)
- Размер: любой (на ёмкость влияет только размер палитры)

4.2 Требования к сообщению

- Кодировка: **UTF-8**
- Максимальная длина: ограничена ёмкостью палитры
- Тип данных: **текст**
- Максимальная теоретическая длина заголовка: 65 535 байт (`uint16`), но на практике ограничена размером палитры

5. Детальное описание алгоритма LSB в палитре PNG

5.1 Структура PNG и чанк PLTE

PNG-файл состоит из последовательности **чанков** (chunks). Каждый чанк имеет структуру:

[4 байта: длина данных] [4 байта: тип чанка] [N байт: данные] [4 байта: CRC]

Для палитрового PNG ключевые чанки:

IHDR — заголовок (ширина, высота, глубина цвета, `ColorType=3`, ...)
 PLTE — палитра: от 1 до 256 записей по 3 байта (R, G, B)
 IDAT — сжатые пиксельные данные (каждый пиксель = индекс в палитре)
 IEND — маркер конца файла

Чанк PLTE — это и есть носитель скрытой информации. Каждая запись палитры хранит тройку (R, G, B), где каждый компонент занимает 1 байт (значение 0–255).

5.2 Принцип LSB

В каждом байте RGB-компонента есть 8 битов: от **бита 7** до **бита 0**. Изменение LSB меняет число на ± 1 :

Пример:

Исходный байт:	10110110	= 182	
Изменение LSB:	10110111	= 183	(+1)
Изменение LSB:	10110110	= 182	(без изменений)

Алгоритм заменяет LSB каждого байта R, G, B в каждой записи палитры на соответствующий бит скрываемого сообщения. Таким образом, каждая запись палитры вмещает **3 бита** скрытой информации.

5.3 Схема распределения битов по палитре

Битовый поток скрытых данных: $b_0 \ b_1 \ b_2 \ b_3 \ b_4 \ b_5 \ b_6 \ b_7 \ b_8 \ \dots$

Палитра:

```
Color[0]:  R0 → LSB = b0
           G0 → LSB = b1
           B0 → LSB = b2
Color[1]:  R1 → LSB = b3
           G1 → LSB = b4
           B1 → LSB = b5
Color[2]:  R2 → LSB = b6
           G2 → LSB = b7
           B2 → LSB = b8
...

```

Биты идут **от старшего к младшему** внутри каждого байта данных.

5.4 Работа с Qt: представление цвета QRgb

В PyQt5 цвет палитры хранится в формате QRgb — 32-битное целое число:

Формат: 0xAARRGGBB

```
Биты 31..24 → AA (альфа-канал, всегда 0xFF для непрозрачного)
Биты 23..16 → RR (красный канал, 0..255)
Биты 15..8  → GG (зелёный канал, 0..255)
Биты 7..0   → BB (синий канал, 0..255)

```

Для работы с каналами реализованы вспомогательные функции:

```
def qRed(rgb: int) -> int:
    return (rgb >> 16) & 0xFF          # Извлечь красный канал

def qGreen(rgb: int) -> int:
    return (rgb >> 8) & 0xFF           # Извлечь зелёный канал

def qBlue(rgb: int) -> int:
    return rgb & 0xFF                  # Извлечь синий канал

def qRgb(r: int, g: int, b: int) -> int:
    return (0xFF << 24) | (r << 16) | (g << 8) | b  # Собрать цвет

```

6. Формат встраиваемых данных

Перед встраиванием текстовое сообщение преобразуется в **бинарный поток** следующей структуры:

2 байта : длина сообщения
N байт : тело сообщения в кодировке UTF-8

6.1 Заголовок длины

Первые 2 байта кодируют длину сообщения как **беззнаковое 16-битное целое в формате big-endian** (сначала старший байт):

```
length_bytes = struct.pack('>H', len(msg_bytes))
# Диапазон: 0..65535
```

Заголовок длины необходим для точного определения границы сообщения при извлечении — без него алгоритм не знает, где заканчивается полезная нагрузка.

6.2 Тело сообщения

Текст кодируется в **UTF-8**, что позволяет работать с любым Unicode-текстом.

```
msg_bytes = message.encode('utf-8')
```

6.3 Итоговый поток данных

```
data = bytearray()
data.extend(struct.pack('>H', length))    # 2 байта заголовка
data.extend(msg_bytes)                  # N байт текста

# Итого: (2 + N) * 8 бит должны поместиться в len(palette) * 3 бит
```

7. Алгоритм встраивания

7.1 Входные данные

- `original_image` — исходное PNG-изображение формата `QImage.Format_Indexed8`
- `message` — текстовая строка для встраивания

7.2 Выходные данные

- `stego_image` — копия исходного изображения с модифицированной палитрой

7.3 Описание алгоритма

Алгоритм встраивания:

Вход:

`image` — `QImage (Format_Indexed8)`
`message` — строка (UTF-8)

1. Закодировать сообщение:
 `msg_bytes` ← `message.encode('utf-8')`
 `length` ← `len(msg_bytes)`
2. Сформировать бинарный поток данных:
 `data` ← `struct.pack('>H', length) + msg_bytes`
 (2 байта заголовка + N байт текста)
3. Проверить ёмкость палитры:
 `colors` ← `image.colorTable()`

```

max_bits ← len(colors) × 3
need_bits ← (2 + length) × 8
ЕСЛИ need_bits > max_bits:
    ОШИБКА "Сообщение не помещается в палитру"

```

4. Создать рабочую копию:

```

img ← image.copy()
colors ← img.colorTable()

```

5. LSB-встраивание:

```

bit_index ← 0
total_bits ← len(data) × 8

```

Для i = 0 до len(colors)-1:

```

r ← qRed(colors[i])
g ← qGreen(colors[i])
b ← qBlue(colors[i])

```

ЕСЛИ bit_index < total_bits:

```

byte_idx ← bit_index // 8
bit_in_byte ← 7 - (bit_index % 8)
bit ← (data[byte_idx] >> bit_in_byte) & 1
r ← (r & 0xFE) | bit
bit_index ← bit_index + 1

```

ЕСЛИ bit_index < total_bits:

(аналогично для g)
bit_index ← bit_index + 1

ЕСЛИ bit_index < total_bits:

(аналогично для b)
bit_index ← bit_index + 1

colors[i] ← qRgb(r, g, b)

ЕСЛИ bit_index ≥ total_bits: ПРЕРВАТЬ ЦИКЛ

6. Применить изменённую палитру:

```

img.setColorTable(colors)

```

7. Сохранить результат:

```

stego_image ← img

```

Выход: stego_image

7.4 Ключевые операции над битами

Замена LSB выполняется двумя битовыми операциями:

```

# Шаг 1: обнулить LSB (маска 0xFE = 11111110)
r_cleared = r & 0xFE

```

```

# Шаг 2: установить нужный бит
r_new = r_cleared | bit # bit ∈ {0, 1}

```

Извлечение бита из потока данных:

```

byte_idx = bit_index // 8 # В каком байте находится нужный бит
bit_in_byte = 7 - (bit_index % 8) # Смещение от LSB (7 = MSB, 0 = LSB)
bit = (data[byte_idx] >> bit_in_byte) & 1 # Извлечённый бит

```

8. Алгоритм извлечения

8.1 Входные данные

- `stego_image` или `original_image` — PNG-изображение с возможно встроенным сообщением

8.2 Выходные данные

- `extracted_message` — строка, извлечённая из LSB-битов палитры

8.3 Описание алгоритма

Алгоритм извлечения:

Вход:

`image` — QImage (Format_Indexed8), содержащее стего-данные

1. Получить палитру:
`colors ← image.colorTable()`
2. Собрать битовый поток из LSB:
`bits ← []`
для КАЖДОГО `color` в `colors`:
 `bits.append(qRed(color) & 1)` // LSB красного
 `bits.append(qGreen(color) & 1)` // LSB зелёного
 `bits.append(qBlue(color) & 1)` // LSB синего

// Итого: `len(colors) × 3` бит
3. Преобразовать биты в байты:
`data_bytes ← []`
для `i = 0` ШАГОМ 8 ДО `len(bits)-7`:
 `byte_val ← 0`
 для `j = 0` ДО 7:
 `byte_val ← (byte_val << 1) | bits[i + j]`
 `data_bytes.append(byte_val)`
4. Считать заголовок длины (первые 2 байта):
`length ← struct.unpack('>H', data_bytes[0:2])[0]`

ЕСЛИ `length > len(data_bytes) - 2`:
 ОШИБКА "Некорректные данные / изображение не содержит сообщения"
5. Извлечь тело сообщения:
`msg_bytes ← data_bytes[2 : 2 + length]`
`message ← msg_bytes.decode('utf-8')`
6. Отобразить результат:
`extracted_output ← message`

Выход: `message`

8.4 Приоритет источника изображения

При извлечении используется следующая логика выбора источника:

```
target = self.stego_image if self.stego_image is not None else  
self.original_image
```


Это позволяет извлекать сообщения как из только что созданного стего-изображения, так и из загруженного с диска PNG-файла.

9. Расчёт ёмкости канала

9.1 Формула ёмкости

Максимальная ёмкость (в битах):

```
max_bits = N_colors × 3
```

Максимальная ёмкость (в байтах):

```
max_bytes = (N_colors × 3) / 8 = N_colors × 0.375
```

Максимальная длина текста (в байтах):

```
max_text = max_bytes - 2 (минус 2 байта заголовка)
```

9.2 Таблица ёмкости для разных размеров палитры

Размер палитры	Бит всего	Байт всего	Максимум текста (ASCII)	Максимум текста (кириллица)
8 цветов	24 бит	3 байта	1 символ	невозможно
16 цветов	48 бит	6 байт	4 символа	2 символа
32 цвета	96 бит	12 байт	10 символов	5 символов
64 цвета	192 бит	24 байт	22 символа	11 символов
128 цветов	384 бит	48 байт	46 символов	23 символа
256 цветов	768 бит	96 байт	~94 символа	~47 символов

9.3 Стандартная конфигурация

Для стандартной 256-цветной палитры:

```
max_bits = 256 × 3 = 768 бит  
max_bytes = 768 / 8 = 96 байт  
max_text = 96 - 2 = 94 байта
```

При ASCII-тексте (1 байт/символ): до 94 символов

При кириллице (2 байта/символ): до 47 символов

При китайском (3 байта/символ): до 31 символа

9.4 Проверка ёмкости

```
max_bits = len(colors) * 3
```

```
needed_bits = (2 + length) * 8
```

```
if needed_bits > max_bits:
```

```
    available_text_bytes = max(0, max_bits // 8 - 2)
```

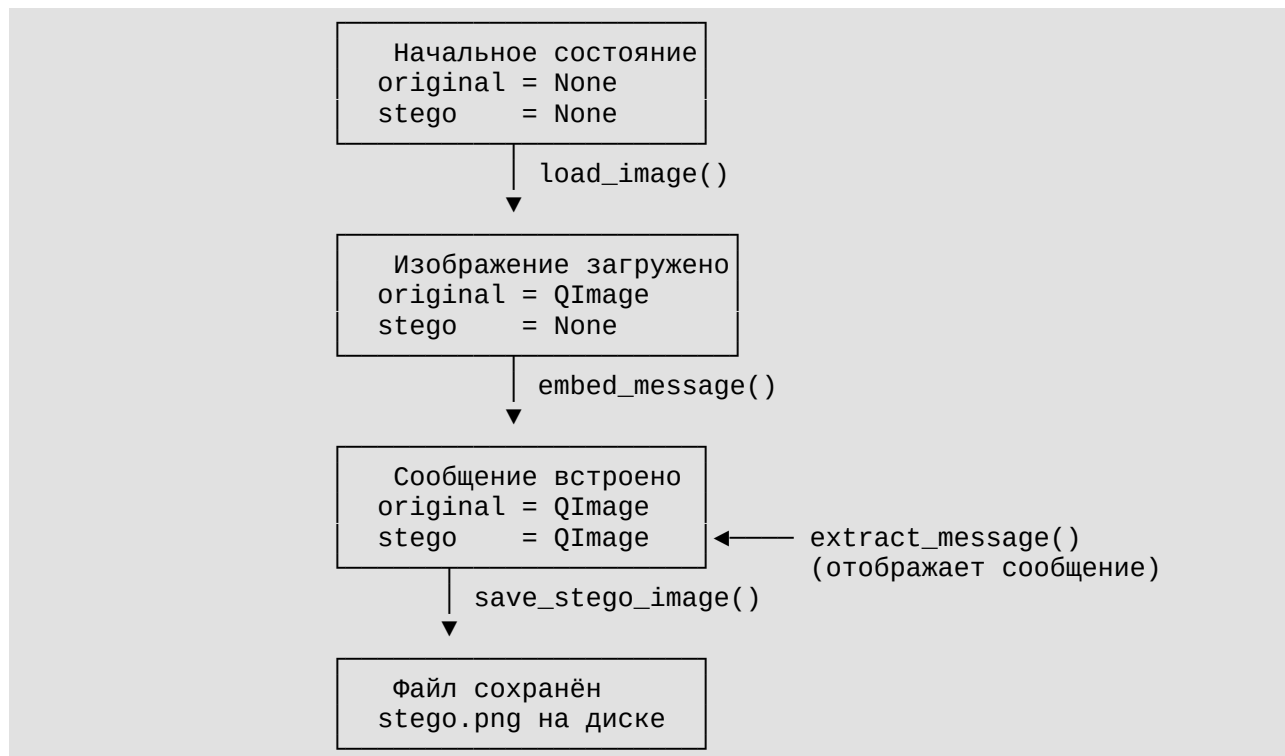
```
    # Показываем ошибку с информацией
```

10. Архитектура приложения

10.1 Структура файлов

```
steganography/
├── main.py          – Основная программа
│   ├── Вспомогательные функции: qRed, qGreen, qBlue, qRgb
│   ├── Константа: APP_STYLE (таблица QSS-стилей)
│   └── Класс: Steganowindow(QMainWindow)
│       ├── __init__()          – инициализация окна
│       ├── _build_ui()         – построение интерфейса
│       ├── _make_header()      – шапка приложения
│       ├── _make_original_group() – панель исходного изображения
│       ├── _make_stego_group() – панель стего-изображения
│       ├── _make_input_group() – поле ввода сообщения
│       ├── _make_output_group() – поле вывода сообщения
│       ├── _make_action_row()  – кнопки действий
│       ├── _refresh_buttons()  – управление состоянием кнопок
│       ├── load_image()        – загрузка PNG из файла
│       ├── save_stego_image()  – сохранение PNG в файл
│       ├── embed_message()     – алгоритм встраивания
│       ├── extract_message()   – алгоритм извлечения
│       └── _show_image()       – отображение QImage в QLabel
├── tests.py          – Тесты
├── requirements.txt  – Зависимости
├── images/           – Тестовые изображения
├── win/              – Собранный .exe для Windows
└── README.md         – Документация проекта
```

10.2 Схема состояний приложения



10.3 Управление состоянием кнопок (`_refresh_buttons`)

Кнопки блокируются/разблокируются автоматически в зависимости от наличия

изображений:

Кнопка	Условие активации
Загрузить	Всегда активна
 Встроить	<code>original_image is not None</code>
 Извлечь	<code>original_image is not None OR stego_image is not None</code>
Сохранить	<code>stego_image is not None</code>

11. Описание тестового покрытия

Файл `tests.py` содержит тесты с маркерами `unit`, `ui`, `integration`, `edge_cases`, `performance`.

11.1 Обзор групп тестов

Группа	Класс	Тип	Описание
1	TestColorHelper Functions	unit	Функции qRed/qGreen/qBlue/qR gb
2	TestLSBAlgorith mCore	unit	Чистая логика LSB без Qt
3	TestUIInitializ ation	ui	Начальное состояние окна
4	TestButtonState s	ui	Логика блокировки кнопок
5	TestEmbedMessag e	ui	Метод embed_message()
6	TestExtractMess age	ui	Метод extract_message()
7	TestFullEmbedEx tractCycle	integration	Полный цикл embed → extract
8	TestFileOperati ons	integration	Сохранение и загрузка файлов
9	TestEdgeCases	edge_cases	Граничные значения
10	TestErrorHandli ng	ui	Обработка ошибочных состояний
11	TestUnicodeMess ages	unit	Unicode, UTF-8, многобайтовые
12	TestPerformance	performance	Скорость операций

11.2 Запуск тестов

```
# Установить зависимости
pip install -r requirements.txt

# Все тесты
pytest tests.py -v

# Только юнит-тесты
pytest tests.py -v -m unit

# Только UI-тесты
pytest tests.py -v -m ui

# Только интеграционные
pytest tests.py -v -m integration

# С кратким выводом ошибок
pytest tests.py -v --tb=short

# С отчётом о покрытии
pytest tests.py -v --cov=main --cov-report=term-missing
```

12. Глоссарий

Термин	Определение
Стеганография	Наука о скрытой передаче информации путём сокрытия самого факта её существования
LSB	Least Significant Bit — наименее значащий (младший) бит числа
PNG	Portable Network Graphics — формат изображений с поддержкой палитры и без потерь
PLTE	Palette chunk — чанк палитры в PNG-файле, содержащий до 256 записей RGB
ColorType = 3	Тип цвета PNG, обозначающий палитровое изображение
Format_Indexed8	Константа Qt, обозначающая 8-битное палитровое изображение
QRgb	32-битное целое Qt для хранения цвета в формате 0xAARRGGBB
Стего-изображение	Изображение со встроенным скрытым сообщением
Носитель	Исходное изображение, в которое встраивается информация
Ёмкость канала	Максимальный объём информации, который можно скрыть в носителе
UTF-8	Кодировка Unicode с переменной длиной

Термин	Определение
	символа (1–4 байта)
Big-endian	Порядок байт, при котором старший байт записывается первым
uint16	Беззнаковое 16-битное целое
Квантование цветов	Уменьшение числа цветов изображения до ≤ 256 для создания палитры
Медианное разрезание	Алгоритм квантования цветов, используемый Qt при конвертации в Indexed8
PyQt5	Python-привязка к библиотеке Qt 5 для создания GUI-приложений
PyInstaller	Инструмент для упаковки Python-приложений в исполняемые файлы
pytest	Фреймворк для тестирования Python-кода
pytest-qt	Плагин pytest для тестирования PyQt5-приложений
QSS	Qt Style Sheets — язык стилизации виджетов Qt

*Документ подготовлен в рамках учебного проекта. Версия 1.0.